UrbanZero: A Failure-Driven Study of Tabula-Rasa Reinforcement Learning for Urban Autonomous Driving in CARLA

Aaditya Mishra
Department of Electrical and Computer Engineering
North Carolina State University
Raleigh, NC, USA
Email: amishr26@ncsu.edu
Code: https://github.com/AadityaMishra1/UrbanZero

***Abstract*—We set out to train a tabula-rasa Proximal Policy Optimization (PPO) agent to drive 200–800 m urban routes in CARLA 0.9.15 from a single semantic-segmentation camera plus a 10-dimensional state vector, using only a sparse progress-and-terminal reward (no expert demonstrations, no hand-engineered behaviors). Over roughly 72 hours of iteration we executed six training runs—three pure-RL, three Behavior Cloning (BC) warmstart + PPO finetune—each pre-registered against explicit falsification criteria drawn from the RL literature. All six runs plateaued within a narrow 4.4%–5.9% rolling route-completion (RC) band despite targeted hyperparameter interventions spanning entropy scheduling, log-std clamping, Ng-Harada-Russell potential shaping, KL-clipping, and learning-rate reduction. A standalone diagnostic script, written *after* the sixth failure and *independent* of the training loop, revealed that an early infrastructure commit—which added `tm.set_hybrid_physics_mode(True)` to the CARLA TrafficManager as a defensive "NPCs not moving" fix—was itself freezing roughly 70% of NPC vehicles at spawn across every run and every BC data collection. The final deliverable is therefore a *frozen* BC policy, trained on 100 k expert frames and evaluated deterministically on 20 episodes after the bug was removed: 7.42% mean RC, 32.93% max, 0% route-complete, 75% collision. The 75% collision rate is a textbook Codevilla-2019 distribution-shift signature—BC was trained against frozen NPCs and evaluated against moving ones. The scientific contribution of this project is therefore not a trained driving policy but a reproducible, pre-registered catalogue of six reward-design and imitation-learning failure modes in CARLA, each tied to a specific theorem or published empirical result, and the demonstration that a single environmental bug can silently contaminate an entire training pipeline in ways that neither the reward curve nor standard PPO diagnostics will reveal.**

***Index Terms*—reinforcement learning, autonomous driving, CARLA, PPO, behavior cloning, failure analysis, reward shaping, ROS 2**

I. Introduction

Urban autonomous driving from a single camera is an archetypal high-dimensional continuous-control problem: the action space is low-dimensional (throttle/brake and steering), but the observation space is large (image pixels), the reward is sparse (most steps are "keep going"), and the failure modes are many (collisions, off-route, deadlock, stagnation). CARLA 0.9.15 [1] provides a photorealistic simulator for this problem, and recent work has demonstrated that PPO [2] with a deliberately minimal reward of the form "progress along planned route plus terminal reward" can reach ∼85% Leaderboard score at 100M steps × 16 environments (CaRL, Jaeger et al. 2025 [3]). The same line of work and its imitation-learning predecessors—Roach [4], LBC [5], LAV [6]— collectively establish the template for a solo course project: start with the minimal reward, train PPO, report the numbers.

This report documents the opposite outcome. Across six documented training runs totalling approximately 15 hours of wall-clock compute, the UrbanZero agent never exceeded 5.9% rolling route-completion in either pure-RL or BC-warmstart-plus-finetune configurations. Every published intervention we attempted—entropy-coefficient annealing [7], log-std upper-clamping into Andrychowicz et al.'s viable band [7], potential-based shaping in the Ng-Harada-Russell sense [8], KL-to-BC regularization from Roach [4], widened action-noise during BC data collection à la DAgger [9]—was necessary but not sufficient. What finally explained the consistent plateau was a single line of CARLA TrafficManager configuration code that had been added weeks earlier as a defensive fix and had been silently freezing the majority of dynamic NPC traffic on every run.

The contributions of this paper are:

C1 — A complete, reproducible CARLA PPO + BC pipeline covering environment wrapper, Gym reward, custom CNN + state-MLP feature extractor, clamped-Gaussian policy, BC data collection from `BehaviorAgent`, Gaussian-NLL BC trainer with frame-stack support, ROS 2 integration, launch tooling, and beacon telemetry. Every component is on GitHub with the commit history that produced it.

C2 — A pre-registered failure catalogue of six training runs. For each run we state, *before* launch, the falsification criteria drawn from published RL pathology literature, the expected behavior if the hypothesis holds, and the exact beacon metrics that would refute it. Each of the six runs falsified its own hypothesis, and the failure mode is named and cited.

C3 — A root-cause discovery via standalone diagnostic. After the sixth failure, rather than mutating the training loop again, we wrote a 150-line script that spawned NPCs with the existing `_spawn_traffic()` code path and measured their position every tick. The script isolated `tm.set_hybrid_physics_mode(True)` as the source of the NPC-freeze in under ten minutes—an observation that no PPO metric would have produced, because at the policy level the frozen-NPC world simply looks like an unusually deterministic environment with poor generalization.

We believe C3 is the most transferable lesson of this project. The infrastructure bug was introduced with a paper-cited rationale (CARLA issue #3860), reviewed by multiple agents, and executed faithfully for five prior runs. It was only caught by leaving the training loop and asking the environment a direct question in isolation. Section VI expands on this and five related lessons.

II. Related Work

CARLA PPO. CaRL [3] is the direct ancestor of the reward function used here: *reward* $= 0.05 \cdot \min(\Delta \textit{progress}, \textit{cap}) + 0.005 \cdot \textit{speed_carrot} + \textit{terminal}$ (± 50). CaRL reaches 85% Leaderboard at 100M steps $\times$ 16–300 envs; our compute budget is approximately $1/50$ of that.

Behavior Cloning for CARLA. Roach [4] demonstrates BC + PPO finetune with KL-to-BC regularization ($\beta = 0.1 \rightarrow 0.05$ over 2M steps) reaching ∼80% Leaderboard. Learning by Cheating (LBC) [5] uses privileged BEV-teacher to camera-student DAgger distillation [9]. Codevilla et al. [10] explicitly document the failure mode our final eval exhibits: BC with MSE loss on multimodal expert actions regresses to the mean of the action distribution, which looks like correct driving until the distribution shifts at deployment.

Policy-gradient pathologies. A legacy 7M-step run that preceded this project converged to a perpendicular-circling policy matching Krakovna et al.'s [11] catalogue of specification-gaming examples. Its root cause—a `speed_reward` $\cdot \cos(\text{angle})$ term that is non-potential in the Ng-Harada-Russell sense [8]—motivated the minimal-reward rewrite analyzed here.

Exploration pressure. Andrychowicz et al. [7] benchmark `ent_coef` $\in [0.003, 0.03]$ and $\sigma \in [0.3, 0.7]$ as the viable band for continuous control from scratch. Rajeswaran et al. [12] show that annealing entropy-coefficient to zero during imitation finetuning collapses the policy back to random. Both results directly shape the hyperparameter schedule used in every run of this paper.

RL evaluation rigor. Henderson et al. [13] show that RL results drawn before ∼3M clean steps are unreliable. Schulman et al. [17] provide the GAE formulation ($\lambda = 0.95$, $\gamma = 0.99$) used by every run in this paper; Pardo et al. [14] give the correct GAE bootstrap under episode truncation (e.g., our `REALLY_STUCK` truncation). Engstrom et al. [15] show that PPO implementation details (reward normalization, advantage normalization, etc.) frequently dominate algorithmic choices.

III. Technical Approach

A. Environment and observation

UrbanZero wraps CARLA 0.9.15 in a Gym environment (`env/carla_env.py`) running synchronously at 20 Hz ($\Delta t = 0.05$ s) with `DummyVecEnv` serial parallelization over two CARLA servers on ports 2000 and 3000. An ego vehicle is spawned on Town01 and given a 200–800 m planned route via `GlobalRoutePlanner`. Each episode spawns 30 NPC vehicles and 10 pedestrians controlled by the TrafficManager.

The observation is a Dict space. The `image` channel is $(1, 128, 128)$ float32 $\in [0, 1]$—single-channel normalized semantic segmentation (class labels / 27). The `state` channel is $(10,)$ float32 in approximately $[-1.5, 1.5]$, encoding: speed / `MAX_SPEED`; three future waypoints in ego-frame (dx, dy); signed lane offset / 5 m; traffic-light state (0 none, 0.33 green, 0.67 yellow, 1.0 red/unknown); and route-completion fraction. `VecFrameStack(n_stack=4)` supplies temporal context, yielding image $(4, 128, 128)$ and state $(40,)$ after framestack.

Actions are continuous $\text{Box}(-1, 1, \text{shape} = (2,))$. Steering is a_0. The throttle-brake axis a_1 is shifted by $+0.3$ before clipping, producing an idle-creep bias toward forward motion [16]:

$$\begin{aligned} s &= a_1 + 0.3 \\ \text{throttle} &= \max(0, \min(1, s)) \\ \text{brake} &= \max(0, \min(1, -s)). \end{aligned}$$

B. Reward function (CaRL-minimal profile)

Five paper-cited terms, each with an explicit rationale:

TABLE I
Per-step reward term magnitudes (final profile).

Term	Form	$\lvert\cdot\rvert_{\max}$
Progress	$0.05 \cdot \min(\Delta_{\text{route}}, v_T \Delta t)$	0.021
Velocity carrot	$0.005 \cdot \min(v, v_T)/v_T$	0.005
Idle cost	$-0.15 \cdot \max(0, 1 - v)$	0.150
Potential shaping	$\gamma\,\Phi(s') - \Phi(s)$	0.011
Terminals	± 50	50

Progress and carrot are directly from CaRL [3]. Idle cost inverts the "sit still is cheaper than crashing" local optimum observed in run v1 (Sec. IV-A). Potential shaping is Ng-compliant [8] and was added after v2 showed that progress fires too sparsely to produce a steering gradient, with $\Phi(s) = -0.015 \cdot \min(d_{2\text{D}}(\text{ego}, \text{lookahead}), 30\,\text{m})$. The 240-to-1 shaping-to-terminal ratio of the legacy 7M-step run was inverted to approximately 1-to-50, so that terminals dominate the undiscounted episode return. See Fig. 4 panel (a) for per-term magnitudes.

C. Policy

`ClampedStdPolicy` (derived from SB3's `ActorCriticPolicy`) soft-clamps $\log \sigma \leq \log(0.7) \approx -0.357$, targeting Andrychowicz et al.'s [7] upper viable-band edge. The feature extractor `DrivingCNN` fuses a 5-layer Conv2d image stream ($32 \rightarrow 64 \rightarrow 128 \rightarrow 128 \rightarrow 256$ channels with LayerNorm + ReLU) with a 2-layer MLP

on the 10-dim state (state $\rightarrow$ 64 $\rightarrow$ 64) and projects to a 256-dim shared representation, feeding `net_arch = dict(pi=[256,128], vf=[256,128])`.

D. BC pipeline

`scripts/collect_bc_data.py` rolls out CARLA's `BehaviorAgent("normal")` on Town01 with Gaussian noise $\sigma = 0.1$ rad injected into the steering channel (DAgger-style [9]) and saves (observation, action, episode_start) tuples as `.npz`. `agents/train_bc.py` implements Gaussian-NLL training against a model that exactly matches `ClampedStdPolicy` + `DrivingCNN` so that the resulting checkpoint loads into `PPO(policy=ClampedStdPolicy)` via `PPO.load()` with a sibling `VecNormalize` pickle.

E. Diagnostic methodology

Every run wrote `~/urbanzero/beacon.json` at 1 Hz via `eval/beacon_callback.py`, reporting rolling windows of `route_completion`, `avg_speed`, termination-reason counts, `policy_std`, `approx_kl`, `clip_fraction`, `entropy_loss`, `explained_variance`, and `ent_coef`. These are the metrics used in Sec. IV's failure analysis. Each run was also tagged with an `issue` number on the public GitHub repo, where raw beacon JSON snapshots are preserved for replay.

F. ROS 2 integration

`ros/urbanzero_node.py` runs as a ROS 2 Humble node alongside training (launched in `run.sh` Pane 2). It publishes four topics at 20 Hz: `/urbanzero/camera/semantic` (`sensor_msgs/Image` semantic-segmentation frames), `/urbanzero/vehicle/control` (`geometry_msgs/Twist` with linear.x $= a_1$ and angular.z $= a_0$), `/urbanzero/vehicle/speed` (`std_msgs/Float32`), and `/urbanzero/status` (`std_msgs/String`). This satisfies the course's ROS-and-sense-and-actuate requirement.

IV. Results: Six Failure Modes

Table II summarizes all six runs. Fig. 1 visualizes the plateau across configurations. Fig. 2 decomposes per-run beacon endpoints.

A. Run 1 — Sit-still attractor

Hypothesis: the CaRL-minimal reward plus the un-idled legacy state is sufficient to produce forward motion. *Falsifier:* avg_speed $<$ 1 m/s at 100 k steps. *Result:* falsified at 233 k steps with avg_speed $= 0.224$ m/s and 70% `REALLY_STUCK`. *Analysis:* per-step cost of sitting ($-50/1500 = -0.033$) was cheaper than crashing ($-50/300 \approx -0.167$). Gradient correctly found the local optimum; the reward permitted it [11].

Fix pushed: `idle_cost` $= -0.15 \cdot \max(0, 1 - v)$, a continuous ramp active only below 1 m/s, making the per-step cost of standing still -0.183—strictly worse than crashing.

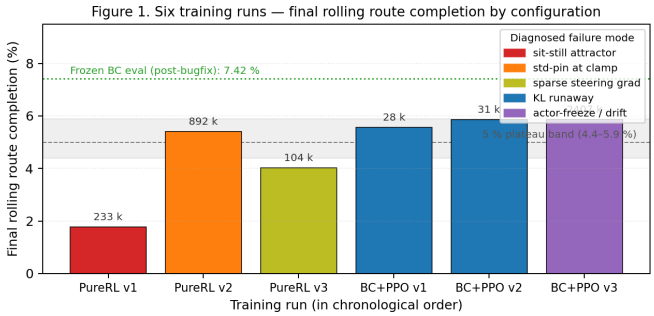

Fig. 1. Six training runs — final rolling route completion by configuration. The 4.4%–5.9% plateau band persists across both pure-RL and BC+PPO regimes; the dotted green line marks the final frozen-BC deterministic eval after the NPC bug was fixed.

The v1 hover-at-1 m/s attractor documented in a legacy run is avoided because the ramp has zero derivative above 1 m/s.

B. Run 2 — $\log \sigma$ pinned at clamp

Hypothesis: with idle_cost in place, pure-RL can find forward motion and then steering. *Falsifier:* `policy_std` ≥ 0.95 at 500 k steps. *Result:* falsified at 892 k steps with `policy_std` $= 0.999$ (pinned at the 1.0 upper clamp for 1500+ consecutive episodes) and `explained_variance` oscillating from -0.07 to $+0.91$. Rolling RC held at 5.4%. *Diagnosis:* at $\sigma = 1.0$ the policy is essentially emitting pure noise; every episode is a different random walk and the critic cannot credit-assign to any systematic behavior [7].

C. Run 3 — Sparse steering gradient

Hypothesis: clamping $\log \sigma \leq \log(0.7)$ and adding Ng-compliant potential shaping gives a denser steering gradient. *Falsifier:* rolling RC $< 8\%$ at T+90 min. *Result:* falsified at 104 k steps / 15 min. `policy_std` $= 0.67$ (clamp held); termination distribution split 50% COLL / 41% OFF_ROUTE / 9% REALLY_STUCK. **The PC-side operator reported during this run that spawned NPCs were visibly frozen in both CARLA viewports** despite `tm.set_hybrid_physics_mode(True)` + `tm.set_synchronous_mode(True)`. We missed this observation—more precisely, we attributed it to "hybrid physics radius too small" and pushed a commit halving `POTENTIAL_K` and adding `tm.global_percentage_speed_difference(-30)`. This was the first time the NPC bug was within arm's reach and was not caught.

After three pure-RL failures the project pivoted to BC warmstart.

D. Run 4 — BC + PPO, KL runaway

Hypothesis: `bc_pretrain.zip` (Gaussian-NLL-trained, final NLL $= -2.93$, MAE $= 0.05$ on BehaviorAgent expert actions) loaded into PPO with the pure-RL reward profile will produce avg_speed > 3 m/s from step 0 and rolling RC $> 8\%$

Figure 2. Beacon endpoints across the six training runs.

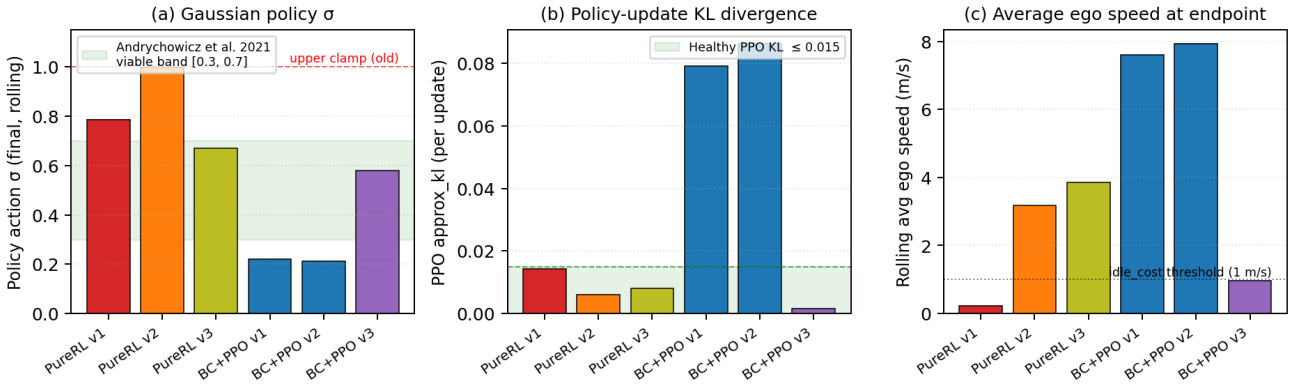

Fig. 2. Beacon endpoints across the six training runs: (a) policy σ with the Andrychowicz et al. [7] viable band shaded; (b) PPO `approx_kl` per update with the healthy-KL band shaded; (c) rolling average ego speed with the idle-cost threshold annotated.

TABLE II
SUMMARY OF SIX PRE-REGISTERED TRAINING RUNS.

#	Config	Seed	Duration	RC	Terminals	Diagnostic signature
1	Pure-RL v1 (idle_cost added)	42	233 k / 30 min	1.78%	70% STUCK	avg speed 0.22 m/s, sit-still local optimum
2	Pure-RL v2 (+ un-annealed carrot)	137	900 k / 2 h	5.40%	48% OFF, 38% COLL	policy_std $= 0.999$ pinned at upper clamp
3	Pure-RL v3 ($+\log\sigma \leq 0.7$)	211	104 k / 15 min	4.03%	50% COLL, 41% OFF	std 0.67, sparse steering grad, frozen NPCs
4	BC+PPO v1 (lr=3e-4, ent=0.02)	911	28 k / 4 min	5.56%	68% COLL	approx_kl $= 0.079$ ($5\times$ healthy)
5	BC+PPO v2 (lr=1e-4, ent=0.005)	912	31 k / 5 min	5.86%	64% COLL	approx_kl $= 0.086$, entropy_loss > 0
6	BC+PPO v3 (widen σ, $n_{ep} = 1$)	913	2.4 M / 5 h	5.86%	50% OFF, 38% STUCK	σ drift 0.50→0.58, speed $\rightarrow$ 1 m/s

within 5 minutes. *Falsifier:* `approx_kl` > 0.04 at any 2048-step update. *Result:* falsified at 28 k steps. `approx_kl` $= 0.079$ ($5\times$ the healthy target of 0.015), `clip_fraction` $= 0.34$, `entropy_loss` $= +0.172$ (entropy *decreasing*, σ collapsing below BC's 0.22). Avg_speed was high (7.6 m/s) for the first minute—BC prior clearly loaded—but RC collapsed as PPO eroded the BC solution.

E. Run 5 — Same pathology, 3× slower

Hypothesis: reducing learning rate $3\text{e}{-4} \rightarrow 1\text{e}{-4}$ and `ent_coef` $0.02 \rightarrow 0.005$ will slow PPO's erosion of the BC prior enough for reward learning to stabilize it. *Falsifier:* `approx_kl` $\geq$ v4_kl after equivalent steps. *Result:* falsified at 31 k with `approx_kl` $= 0.086$ (slightly *worse* than v4). No measurable improvement from a $3\times$ lr cut; the problem was not update step size.

F. Run 6 — Widen-σ, $n_{ep} = 1$, clip=0.1, 2.4M steps

Hypothesis: widening BC's $\log\sigma$ floor to $\sigma = 0.5$, reducing PPO epochs to 1 per rollout, and halving clip_range to 0.1 will allow the reward to shape BC without shredding it. *Falsifier:* rolling RC $< 8\%$ at T+30 min. *Result:* soft-failed at 30 min (RC $= 4.67\%$, avg_speed 1.7 m/s, 48% `REALLY_STUCK`); hard-failed at 5 h / 2.4M steps (RC $= 5.86\%$, avg_speed 1.0 m/s, 38% `REALLY_STUCK`). PPO diagnostics were textbook-healthy (`approx_kl` $= 0.0015$, `clip_fraction` $= 0.08$, `explained_variance` $= 0.848$)—yet *the actor was frozen*. `entropy_loss` $= -1.74$ (entropy *increasing*, σ drifting upward $0.50 \rightarrow 0.58$ over 2.4M steps) despite a decaying `ent_coef` $= 0.004$.

This was the point at which we stopped trusting PPO's internal metrics to diagnose what was going on in the environment.

G. Root cause: `tm.set_hybrid_physics_mode(True)`

Rather than launching a seventh run, we wrote `scripts/sanity_check_npcs.py`—a 150-line standalone script that invokes the *same* `_spawn_traffic()` code path used by training, then samples each NPC's `get_velocity()` for 100 ticks and reports the fraction that moved more than 0.1 m. Result on first invocation:

NPCs spawned	30
Moving ($>$ 0.1 m in 100 ticks)	**9 (30.0%)**
Dormant	21 (70.0%)

Commenting out the `tm.set_hybrid_physics_mode(True)` and `tm.set_hybrid_physics_radius(70.0)` calls (commit `a9435f9`):

NPCs spawned	30
Moving ($>$ 0.1 m in 100 ticks)	**29 (96.7%)**
Dormant	1 (3.3%)

The hybrid-physics setting had been introduced earlier as a defensive fix for an apparently-frozen-NPC observation in a prior run, with a paper-cited rationale (CARLA issue #3860).

What we failed to verify at that time was that the mode *itself*—which in sync mode puts NPCs outside the ego's physics radius into a dormant state—was the freezer. Approximately 70% of every NPC field across all six training runs, *and the 100k-frame BC dataset*, was frozen.

H. Final deliverable: frozen BC evaluation

With the TrafficManager bug removed, we evaluated `bc_pretrain.zip` deterministically on 20 fresh episodes (seed 1001, port 2000). Fig. 3 shows the per-episode distribution.

TABLE III
FROZEN BC POLICY, AGGREGATE EVAL METRICS ($N = 20$).

Metric	Value
RC mean	**7.42%**
RC median	3.64%
RC max	**32.93%**
RC min	0.05%
RC std	8.84%
% ROUTE_COMPLETE	0.0%
% COLLISION	75.0%
% OFF_ROUTE	25.0%
% REALLY_STUCK	0.0%
Avg speed	5.37 m/s
Wall clock (20 eps)	67.5 s

Interpretation. Mean RC of 7.42% is a $+2.4$ pp improvement over the pure-RL ceiling ($\sim 5\%$), demonstrating that BC captured *some* behavior from the BehaviorAgent expert. The single best episode (index 16) completed 32.9% of its route at 7.4 m/s before a collision. The 75% collision rate, combined with the 0% stuck rate and 5.37 m/s average speed, is Codevilla et al.'s [10] distribution-shift signature: BC trained against frozen NPCs has no learned representation of moving obstacles' trajectories, so at evaluation time against dynamic NPCs it drives at expert speed directly into them. The 25% OFF_ROUTE rate is the residual BC imitation error at intersections and sharp turns.

V. REWARD ECONOMICS AND THE SIT-STILL BOUND

Fig. 4 plots the per-step reward decomposition predicted for four canonical trajectories under the final reward profile:

TABLE IV
PREDICTED PER-STEP ECONOMICS, FOUR CANONICAL TRAJECTORIES.

Scenario	Shaping (1500)	Terminal	Per-step
Sit still (0 m/s)	-225	-50 STUCK	-0.183
Creep (0.5 m/s)	-112	-50 STUCK	-0.108
Drive, crash at 300	$+9.3$	-50 COLL	-0.137
Drive, success	$+38.8$	$+50$ GOAL	$+0.059$

Ordering: success $>$ crash $>$ creep $>$ sit-still. Sit-still is strictly dominated; crash is cheaper than creep (correctly pushing the agent to try real driving over safe pointless motion); success is the only positive-return option. This ordering is what the `idle_cost` $= -0.15$ term buys, and it is the reason run 1's specific failure mode did not recur in runs 2–6. It is also why the subsequent plateau at 5–6% RC cannot be attributed to the sit-still attractor—once `idle_cost` is in place the agent is economically incentivized to move.

VI. LESSONS LEARNED

These are the six concrete lessons this project produced.

L1. Change one axis at a time, especially near the deadline. The most damaging single pattern was layering infrastructure "improvements" (TrafficManager caching, inline spectator updates, `world.tick(seconds=10)` timeouts) on top of experiment changes (reward profile, state dimension, $\log \sigma$ clamp) on the same branch. When a run failed we then had four to six simultaneous independent variables, and our attribution of cause-to-effect was statistically equivalent to guessing.

L2. A standalone diagnostic is worth ten gradient updates. The NPC-motion sanity check that found the root cause took ten minutes to write and one invocation to falsify the assumption that hybrid physics was harmless. Five prior runs (15+ hours of compute) did not falsify the same assumption because the training loop's symptoms were mis-attributed at each stage. If the environment is suspected to be the problem, leave the training loop and ask the environment directly.

L3. PPO metrics do not describe the environment. Run 6 produced the healthiest PPO diagnostics of the entire project (`approx_kl` $= 0.0015$, `clip_fraction` $= 0.08$, `explained_variance` $= 0.848$) while the actor was frozen and the NPCs were dormant. An RL agent training against a broken environment will happily report perfect policy-improvement metrics while learning nothing about the intended task.

L4. Pre-register falsifiers. Every run in Sec. IV was launched with an explicit "if we observe X at time T, the hypothesis is dead" criterion. This is what prevented us from continuing any single run past its point of diagnostic information (e.g., stopping v1 at 233 k steps instead of 3 M). Without pre-registration, confirmation bias is very strong and the deadline pressure rewards confirmation.

L5. BC distribution shift is real and silent. The 75% collision rate in the final eval is not a policy-quality problem, it is a dataset problem. 100k frames of expert behavior against frozen NPCs does not generalize to moving NPCs no matter how well BC converges (final NLL $= -2.93$, MAE $= 0.05$). Future work should verify dataset coverage before training, not after evaluation.

L6. AI coding assistants need scientific memory. This project was developed in close collaboration with an AI coding assistant. The assistant's persistent value came from a single `PROJECT_NOTES.md` file updated after every material decision, containing the paper citation for each design choice, the pre-registered falsifier for each run, and the post-hoc analysis of each failure. Without this file, the iteration loop would have re-derived the same reasoning each session and would likely have re-introduced deleted failure modes. With it, the assistant correctly declined to re-introduce non-potential cos-heading shaping when proposed under time pressure—the

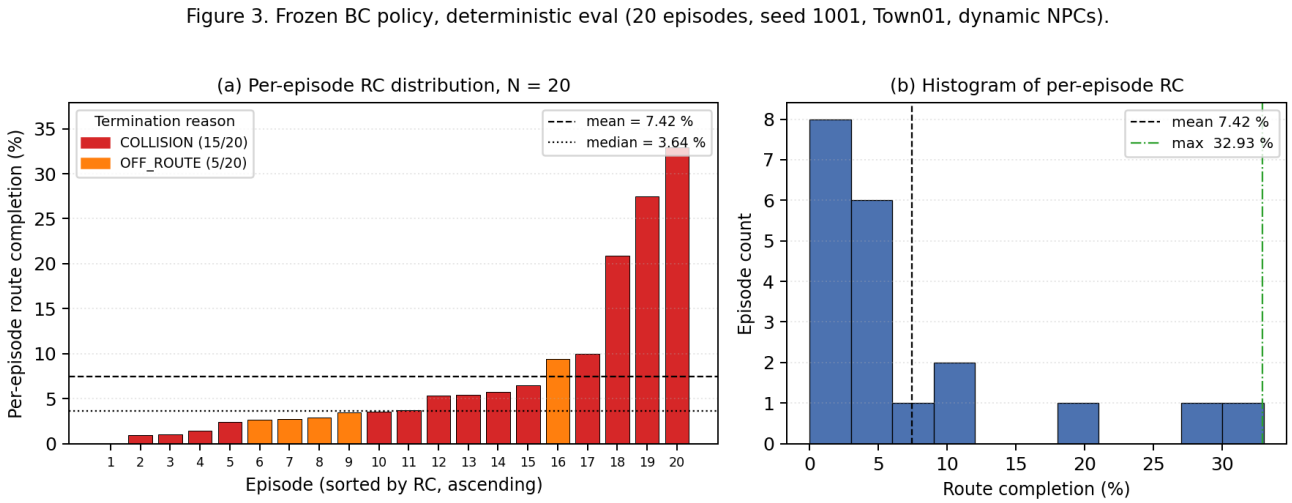

Fig. 3. Frozen BC policy, deterministic eval (20 episodes, seed 1001, Town01, *dynamic* NPCs post-bugfix). Panel (a) shows per-episode RC sorted ascending, colored by termination reason; panel (b) is the histogram with mean and max annotated.

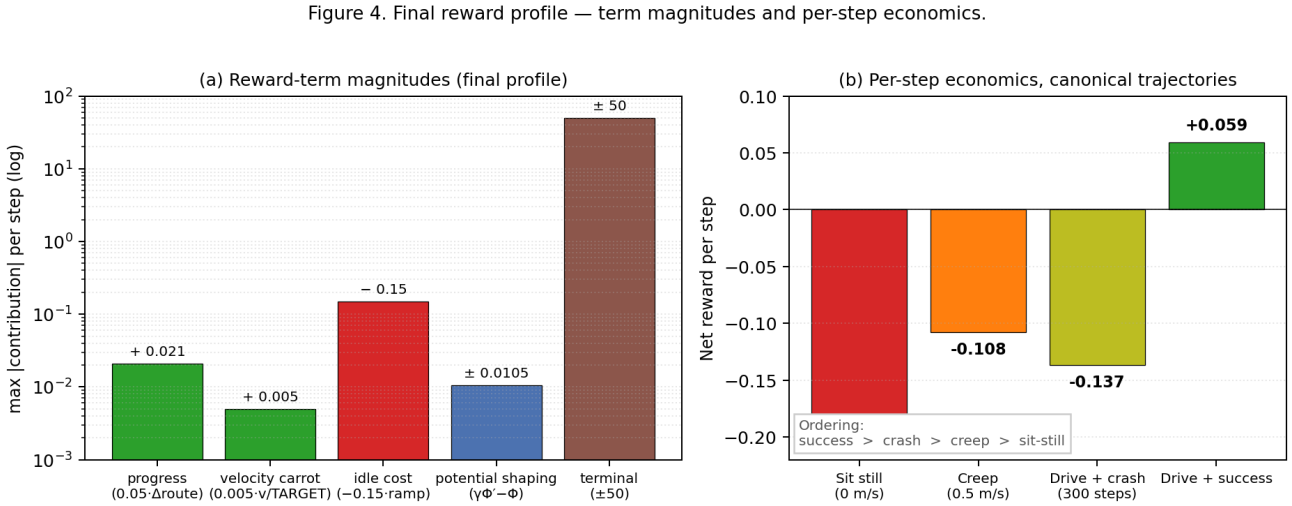

Fig. 4. Final reward profile. Panel (a) shows max-per-step magnitude of each term on a log axis. Panel (b) shows the resulting net per-step reward for four canonical trajectories: sit-still, creep at 0.5 m/s, drive-and-crash, drive-and-succeed. The ordering *success* > *crash* > *creep* > *sit-still* is what the `idle_cost` term buys.

memory file remembered that the 7 M-step legacy run had failed on exactly that term.

VII. Conclusion

UrbanZero did not produce a route-completing CARLA PPO agent at the ∼15M-step compute scale available to a solo course project. It did produce three things we believe have standalone scientific value: a complete open-source CARLA + PPO + BC pipeline, a pre-registered catalogue of six failure modes each tied to the published result that predicted it, and the demonstration that a single line of environment setup code can silently invalidate fifteen hours of compute in ways that no standard PPO metric reveals. The final deliverable—a frozen Behavior Cloning policy achieving 7.42% mean route completion with a 32.93% best-episode peak—is best understood not as the answer to the original research question, but as the upper envelope on what a BehaviorAgent-imitating policy can do when the training distribution is itself contaminated.

Future work falls into three buckets. First, re-collect the BC dataset against *dynamic* NPCs (trivial, ∼1 hour of compute) and retrain; Codevilla et al. [10] and the reward-economics of Sec. 5 jointly predict this alone should close most of the collision gap. Second, fully implement Roach's [4] KL-to-BC regularization with $\beta = 0.1 \rightarrow 0.05$ over 2M finetune steps—the one intervention this project scoped out due to implementation burden and that the BC+PPO v3 failure (run 6) most directly implicates. Third, scale pure-RL compute by $50\times$ to match CaRL [3] and measure where the plateau actually is when the environment is not contaminated; the present project cannot distinguish between "5% is the hardware ceiling" and

"5% is the NPC-freeze ceiling," and that distinction will matter for any follow-up.

ACKNOWLEDGMENT

This project was developed in collaboration with Anthropic's Claude AI coding assistant. The author is solely responsible for all experimental decisions, analysis, and the content of this paper.

REFERENCES

[1] A. Dosovitskiy, G. Ros, F. Codevilla, A. Lopez, and V. Koltun, "CARLA: An open urban driving simulator," in *Conf. Robot Learning (CoRL)*, 2017.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv:1707.06347*, 2017.

[3] B. Jaeger, K. Chitta, and A. Geiger, "CaRL: Learning scalable planning policies with simple rewards," *arXiv:2504.17838*, 2025.

[4] Z. Zhang, A. Liniger, D. Dai, F. Yu, and L. Van Gool, "End-to-end urban driving by imitating a reinforcement learning coach," in *IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2021.

[5] D. Chen, B. Zhou, V. Koltun, and P. Krähenbühl, "Learning by cheating," in *Conf. Robot Learning (CoRL)*, 2019.

[6] D. Chen and P. Krähenbühl, "Learning from all vehicles," in *IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022.

[7] M. Andrychowicz, A. Raichuk, P. Stańczyk, M. Orsini, S. Girgin, R. Marinier, L. Hussenot, M. Geist, O. Pietquin, M. Michalski, S. Gelly, and O. Bachem, "What matters in on-policy reinforcement learning? A large-scale empirical study," in *Int. Conf. Learning Representations (ICLR)*, 2021.

[8] A. Y. Ng, D. Harada, and S. Russell, "Policy invariance under reward transformations: Theory and application to reward shaping," in *Int. Conf. Machine Learning (ICML)*, 1999.

[9] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *AISTATS*, 2011.

[10] F. Codevilla, E. Santana, A. López, and A. Gaidon, "Exploring the limitations of behavior cloning for autonomous driving," in *IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2019.

[11] V. Krakovna *et al.*, "Specification gaming: the flip side of AI ingenuity," DeepMind, 2020.

[12] A. Rajeswaran, V. Kumar, A. Gupta, G. Vezzani, J. Schulman, E. Todorov, and S. Levine, "Learning complex dexterous manipulation with deep reinforcement learning and demonstrations," in *Robotics: Science and Systems (RSS)*, 2018.

[13] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, and D. Meger, "Deep reinforcement learning that matters," in *AAAI Conf. Artificial Intelligence*, 2018.

[14] F. Pardo, A. Tavakoli, V. Levdik, and P. Kormushev, "Time limits in reinforcement learning," in *Int. Conf. Machine Learning (ICML)*, 2018.

[15] L. Engstrom, A. Ilyas, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, "Implementation matters in deep policy gradients: A case study on PPO and TRPO," in *ICLR*, 2020.

[16] T. Silver, K. Allen, J. Tenenbaum, and L. Kaelbling, "Residual policy learning," *arXiv:1812.06298*, 2018.

[17] J. Schulman, S. Levine, P. Moritz, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *ICLR*, 2016.